

Curs 7 — Structuri de Date (Gabriel Istrate)

Sinteză: Geometrie computațională (intersecția segmentelor, convex hull)

🎯 = punct de examen, ⚠️ = capcană, ★ = fapt fundamental.

„Un mic respiro de la structurile de date” — dar algoritmi folosesc fix structurile învățate: **stive** (Graham's scan) și **red-black trees** (intersecția segmentelor). De aici și legăturile cu grila.

0. De ce e relevant de examen

Tip de întrebare	Unde apare	Ce-ți trebuie
Convex hull — „niciun răspuns corect”	model Q13, grila2025 Q14	Graham sortează polar, Jarvis nu sortează
Jarvis's march „trebuie să...”	grila2025 Q14	pleacă din punctul cu cel mai mic y
Intersecția segmentelor — proprietăți	grila2025 Q25	$O(n \log n)$, RB tree, test la insert și delete, sortare
Cine folosește stivă	Doc1 Q11, Q14	Graham's scan folosește stivă

1. Introducere

Geometria computațională studiază algoritmi pentru probleme geometrice. Aplicații: grafică, robotică, VLSI, CAD, plus protein folding, modelare moleculară, GIS. (Conferință: SOCG; bibliotecă: CGAL.)

⚠️ **Inamicul numărul 1: degenerarea** (degeneracy) — trei puncte coliniare, trei drepte care se intersectează în același punct etc. Algoritmi reali au nevoie de „patch-uri” pentru cazurile degenerate, dar în curs sunt ignorate.

Definiții de bază:

- Input: o mulțime de puncte $(p_i = (x_i, y_i))$; un poligon $P = (p_0, p_1, \dots, p_n)$.
- Combinație convexă** a lui p_1 și p_2 : orice punct $p_3 = \lambda \cdot p_1 + (1-\lambda) \cdot p_2$ cu $\lambda \in [0, 1]$ (adică un punct **pe segmentul** dintre p_1 și p_2). E baza noțiunii de convexitate.

2. Produse vectoriale (cross products)

Instrumentul fundamental — un singur calcul $O(1)$ îți spune orientarea.

Produsul vectorial a doi vectori (în 2D):

$$p_1 \times p_2 = \det \begin{pmatrix} x_1 & x_2 \\ y_1 & y_2 \end{pmatrix} = x_1 y_2 - x_2 y_1$$

E **aria cu semn** a paralelogramului format de p_1 și p_2 . Semnul dă orientarea.

La ce folosește:

1. Dat segmentul $(p_0 p_1)$, să afli dacă $(p_0 p_2)$ e **în sens orar (clockwise)** sau **trigonometric (counterclockwise)** față de el.
2. Dat traseul $(p_0 p_1)$ apoi $(p_1 p_2)$, să afli dacă faci **viraj la stânga sau la dreapta** în p_1 .
 - ★ **counterclockwise** $\Leftrightarrow$ **viraj la stânga**; **clockwise** $\Leftrightarrow$ **viraj la dreapta**.
3. Să testezi dacă două segmente se intersectează.

DIRECTION și ON-SEGMENT:

```
DIRECTION(pi, pj, pk):  
    return (pk - pi) × (pj - pi)           // semnul = orientarea virajului  
  
ON-SEGMENT(pi, pj, pk):                    // presupune pk coliniar cu pi-pj  
    if min(xi,xj) ≤ xk ≤ max(xi,xj) and min(yi,yj) ≤ yk ≤ max(yi,yj)  
        return TRUE  
    else return FALSE
```

3. Intersecția a DOUĂ segmente

Două etape:

Etapa 1 — QUICK REJECT (respingere rapidă prin bounding box): două segmente **nu se pot** intersecta dacă **dreptunghiurile lor de încadrare (bounding boxes) nu se intersectează**. Bounding box-ul = cel mai mic dreptunghi care conține segmentul, cu laturile paralele cu axele (colțurile sunt min/max ale coordonatelor capetelor).

Etapa 2 — STRADDLING (segmentul „încalecă” dreapta): un segment $(p_1 p_2)$ încalecă o dreaptă dacă p_1 e de o parte a dreptei și p_2 de cealaltă parte (sau dacă unul din ele e pe dreaptă).

★ Două segmente se intersectează dacă și numai dacă: trec testul de respingere rapidă
ȘI fiecare segment încăleacă dreapta care conține celălalt segment.

```
SEGMENTS-INTERSECT(p1, p2, p3, p4):  
    d1 = DIRECTION(p3, p4, p1)  
    d2 = DIRECTION(p3, p4, p2)  
    d3 = DIRECTION(p1, p2, p3)  
    d4 = DIRECTION(p1, p2, p4)  
    if ((d1>0 and d2<0) or (d1<0 and d2>0)) and  
        ((d3>0 and d4<0) or (d3<0 and d4>0))  
        return TRUE // se încăleacă reciproc  
    elseif d1 == 0 and ON-SEGMENT(p3, p4, p1): return TRUE  
    elseif d2 == 0 and ON-SEGMENT(p3, p4, p2): return TRUE  
    elseif d3 == 0 and ON-SEGMENT(p1, p2, p3): return TRUE  
    elseif d4 == 0 and ON-SEGMENT(p1, p2, p4): return TRUE  
    else return FALSE // cazurile cu d=0 = coliniaritate
```

(Semnele opuse ale lui d_1, d_2 înseamnă că p_1, p_2 sunt de o parte și de alta a dreptei $p_3 p_4$ — adică se încăleacă. Cazurile $d=0$ tratează coliniaritatea.)

4. Intersecția ORICĂROR două segmente (sweep line) — critic de examen

Problema: date n segmente, există vreo pereche care se intersectează? (Doar DA/NU, nu unde.)

Tehnica: sweeping (baleiere). O dreaptă verticală imaginară trece prin obiecte, de obicei de la stânga la dreapta. Dimensiunea pe care se mișcă dreapta (aici x) e tratată ca o dimensiune de timp.

★ **Complexitate:** $O(n \log n)$ (algoritmul naiv, care testează toate perechile, e $O(n^2)$).

Două structuri de date:

1. **Sweep-line status (T):** dă relațiile dintre obiectele intersectate de dreapta de baleiere în acel moment — o **ordine totală**. Operațiile:
 - `INSERT(T, s)`, `DELETE(T, s)`;
 - `ABOVE(T, s)`: segmentul de deasupra lui s ;
 - `BELOW(T, s)`: segmentul de dedesubtul lui s .
 - ★ Fiecare operație se face în $O(\log n)$ folosind red-black trees.
2. **Event-point schedule:** secvența de coordonate x (sortate stânga→dreapta) care definesc pozițiile de oprire ale dreptei = **event points**. ★ Statusul se schimbă DOAR în event points (la capetele segmentelor).

ANY-SEGMENTS-INTERSECT(S):

```
T = ∅
sortează capetele segmentelor stânga→dreapta
    (la egalitate: capetele stângi înaintea celor drepte; apoi după y
    crescător)
for fiecare punct p din lista sortată:
    if p e capătul STÂNG al unui segment s:
        INSERT(T, s)
        if ABOVE(T,s) intersectează s or BELOW(T,s) intersectează s:
            return TRUE // ← test la INSERARE
    if p e capătul DREPT al unui segment s:
        if ABOVE(T,s) și BELOW(T,s) există și se intersectează:
            return TRUE // ← test la ȘTERGERE
        DELETE(T, s)
return FALSE
```

🌀 Exact ce se testează la grila2025 Q25:

- testezi intersecția și când adaugi un capăt (insert), și când ștergi unul (delete) → adevărat;
- complexitate $O(n \log n)$ → adevărat;
- folosești sortarea ca primitivă (sortezi capetele) → adevărat;
- ⚠️ NU folosești un **splay tree**, ci un **red-black tree** (variantea cu splay e falsă).

De ce e corect: sunt $2n$ capete → $2n$ operații insert/delete/test, fiecare $O(\log n)$ → $O(n \log n)$. Algoritmul poate greși doar dacă **nu** raportează o intersecție; demonstrația arată că la prima intersecție (cea mai din stânga) cele două segmente devin consecutive în ordinea T fix la un event point, unde testul ABOVE/BELOW o prinde.

5. Convex hull (înfășurătoarea convexă)

Definiție: cel mai mic poligon convex care conține mulțimea de puncte. (Analogie: pui un elastic în jurul punctelor și îl lași să se strângă.)

Doi algoritmi principali:

- **Graham's Scan:** $O(n \log n)$.
- **Jarvis's March:** $O(n \cdot h)$, unde h = numărul de puncte de pe hull.

Alți algoritmi (doar menționați): incremental (adaugi p_i la $CH(p_1 \dots p_{i-1})$), divide-and-conquer (împarți în jumătatea stângă/dreaptă, calculezi hull-urile și le combini), prune-

and-search.

6. Graham's Scan — critic de examen

★ **Menține o STIVĂ S de puncte candidate.** Fiecare punct e pus pe stivă (push); punctele care nu sunt pe hull sunt eventual scoase (pop). Funcții auxiliare: $\text{TOP}(S)$, $\text{NEXT-TO-TOP}(S)$ (nu modifică stiva). **Returnează punctele hull-ului în ordine trigonometrică (counterclockwise).**

GRAHAM-SCAN(Q):

```
1 fie p0 = punctul cu cel mai mic y (cel mai din stânga la egalitate)
2 fie (p1, ..., pm) = restul punctelor, SORTATE după UNghiUL POLAR
   trigonometric în jurul lui p0
   (la unghi egal, păstrezi doar punctul cel mai depărtat de p0)
3 PUSH(p0, S)
4 PUSH(p1, S)
5 PUSH(p2, S)
6 for i = 3 to m:
7     while unghiul format de NEXT-TO-TOP(S), TOP(S), pi NU e viraj la
stânga:
8         POP(S)
9     PUSH(pi, S)
10 return S
```

Complexitate: linia 1 $\Theta(n)$; sortarea $\Theta(n \log n)$; testul de viraj (produs vectorial) $\Theta(1)$; restul $O(n)$. → **Dominat de sortare: $O(n \log n)$.**

🌀 **De examen:** Graham's scan folosește o **STIVĂ** și **sortează după unghiul polar** în jurul punctului de bază (cel mai de jos). NU folosește coadă, NU sortează după axe.

7. Jarvis's March — critic de examen

★ Folosește **gift wrapping (împachetare cadou)** — simulează înfășurarea unei foi de hârtie în jurul punctelor.

Cum merge:

- Pleacă din **același p_0** ca Graham = punctul cu **cel mai mic y**.
- „Tragi hârtia” la dreapta, apoi tot mai sus, până atinge un punct → acela e un vârf al hull-ului. Continui până te întorci la p_0 .

- Formal: din p_0 alegi $\boxed{p_1}$ = punctul cu **cel mai mic unghi polar** față de p_0 ; apoi $\boxed{p_2}$ = cel mai mic unghi polar față de p_1 ; ... până la cel mai înalt punct p_k . Asta e **lanțul drept (right chain)**. Apoi construiești **lanțul stâng (left chain)** pornind de la p_k și măsurând unghiuri polare față de axa x **negativă**.

Complexitate: $O(n \cdot h)$ — la fiecare din cele h vârfuri ale hull-ului faci o trecere $O(n)$ ca să găsești următorul punct.

🌀 **De examen:** Jarvis pleacă din punctul cu cel mai mic y (asta e răspunsul la grila 2025 Q14). ⚠️ Jarvis **NU folosește stivă** (aia e la Graham) și **NU pre-sortează** toate punctele (alege la fiecare pas minimul de unghi polar, printr-o căutare liniară — de aici $O(n \cdot h)$).

8. Cheat-sheet — fapte de examen

Concept	De reținut
Produs vectorial	$\boxed{x_1 y_2 - x_2 y_1}$; semnul = orientare; CCW = viraj stânga
Două segmente se intersectează	quick reject (bounding box) + fiecare încalcă dreapta celuilalt
Oricare două segmente (sweep line)	$O(n \log n)$ (naiv $O(n^2)$)
Sweep-line status	red-black tree , INSERT/DELETE/ABOVE/BELOW $O(\log n)$
Când testezi intersecția	la inserarea ȘI ștergerea unui capăt
Sweep line — primitivă	sortare a capetelor ($2n$ event points)
Sweep line — NU folosește	splay tree (folosește RB)
Convex hull	cel mai mic poligon convex care conține punctele
Graham's Scan	stivă , sortare după unghi polar , $O(n \log n)$, ieșire CCW
Jarvis's March	gift wrapping , pleacă din min-y , fără pre-sortare, $O(n \cdot h)$
Graham vs axe	sortează după unghi polar, NU după O_x/O_y
Punct de start (ambele)	cel cu cel mai mic y

9. Conexiuni de examen, pas cu pas

Convex hull — „niciun răspuns corect” (model Q13, grila2025 Q14 generic): la întrebarea „când calculăm convex hull-ul, folosim coadă / sortăm după Oy / sortăm după Ox?” răspunsul e **niciunul**, fiindcă:

- **Graham's Scan** folosește o **stivă** (nu coadă) și sortează după **unghiul polar** (nu după axe Ox/Oy);
- **Jarvis's March** nu folosește cozi/stive și **nu pre-sortează**.

Jarvis specific (grila2025 Q14): „când calculăm convex hull cu Jarvis's march trebuie să...” → **calculăm punctul cu cea mai mică coordonată Oy** (punctul de start p_0). ⚠ Celelalte variante („folosim o stivă”, „sortăm după unghi polar”) sunt ale lui **Graham**, nu ale lui Jarvis.

Intersecția segmentelor (grila2025 Q25): testezi la **insert ȘI delete**, e $O(n \log n)$, folosești **sortarea** ca primitivă → variantele (b),(c),(d) adevărate; varianta cu **splay tree** e falsă (e red-black tree).

Cine folosește stivă (Doc1 Q11/Q14): **Graham's Scan** și **DFS** folosesc stivă; **BFS** folosește coadă.

*Bottom line: din acest curs, pentru grilă reține în primul rând distincția convex hull — **Graham's Scan** (stivă + sortare după unghi polar + $O(n \log n)$, ieșire trigonometrică) vs **Jarvis's March** (gift wrapping + pleacă din punctul cu cel mai mic y + $O(n \cdot h)$, fără pre-sortare). Capcana clasică: „convex hull, sortăm după axe / folosim coadă?” → NU (Graham sortează polar, Jarvis nu sortează). Pentru intersecția segmentelor: sweep line, $O(n \log n)$, **red-black tree** (nu splay), testezi la inserarea și ștergerea capetelor, sortarea ca primitivă. Produsul vectorial (semnul = orientare/viraj) e instrumentul $O(1)$ din spatele tuturor.*